PRE-PAPER · WORKING DRAFT

Agentic Weight-Space Programming as an Alternative to Fine-Tuning in Small Language Models

Authors: Luiz Melo · **Affiliation:** Universidade do Sul de Santa Catarina (Unisul) · **Draft:** November 11, 2026

Working draft – status of each result and remaining work are listed at the end.

TL;DR

- **The defect.** Small "thinking" models often get stuck repeating themselves: *"Wait, let me check… Yes… Wait…"*. Out of every 100 grade-school math problems, Qwen3.5-0.8B runs out of space on 85 before writing an answer.
- **The fix.** We gave Claude agents the model's weights and a small testing budget, and asked them to fix this by editing the weights directly, with no training. **With four tiny edits, the model went from solving 15% to 67% of our practice problems, and from 15.5% to 61.5–67.5% of 200 locked test problems.**
- **The comparison.** At our 12k-token budget the agents' edits are ahead of the one published in *Sparks of Weight-Space Programming* (53.5%), but that edit is still improving when it hits the budget. The supported claim is efficiency: the agents' edits reach their plateau with about a third of the tokens. They beat one cheap fine-tuning recipe (29% on practice problems, with 13× more GPU time).
- **The simple alternatives are strong.** Capping thinking time (60–61%), banning the word "Wait" (60.0%) and switching thinking off (58.5%) all tie the first agent's edit on the locked test. Only two things are clearly above thinking off: that edit plus a thinking cap (69.5%) and the second agent's edit (67.5%).
- **What mattered.** Two independent agents found the same "Wait" neurons and the same `</think>` lever, although they pushed the shared neurons in opposite directions. Two later runs with another editor model saw the same neurons and fixed the loops with a different kind of edit. A simple script with the same tools and budget, but no agent reasoning, did not. Every cheap shortcut score we gave the agents was eventually gamed, and only real test runs kept them honest. In three pairs of runs, agents given more interpretability data found worse fixes.
- **Where it stops working.** On a second, diffuse defect (following formatting instructions, IFEval), neither the agents, nor a system prompt, nor fine-tuning improved the model. Every candidate that looked better in practice checks lost 3–7 points on fresh prompts. The method works when the defect is concentrated and its effect is large; it cannot find effects smaller than the test's own sampling noise.

Everything ran on one gaming graphics card (GTX 1660 SUPER, 6 GB).

A small model that loops is usually treated as a training problem: collect examples of good reasoning and fine-tune. That works, but for a looping model even collecting the examples is expensive, because most of its attempts never end.

Recently, Takehi et al. showed something different. A frontier model (GPT-6 "Astra") read the raw weights of this same model and edited 0.016% of them, and MATH-500 accuracy went from 7.9% to 48.1%. We wanted to know three things:

1. **Does it reproduce,** on a cheap setup, and what exactly did that edit do?
2. **Is it more than a trick?** Could a decoding setting or ordinary fine-tuning do the same?
3. **What in the agent's process actually matters?**

The defect: a model that can't stop thinking

Qwen3.5-0.8B "thinks" in a scratchpad before answering and ends its thinking with a special `</think>` marker. With a budget of 12,288 tokens (about 9,000 words), most of its answers never get there.

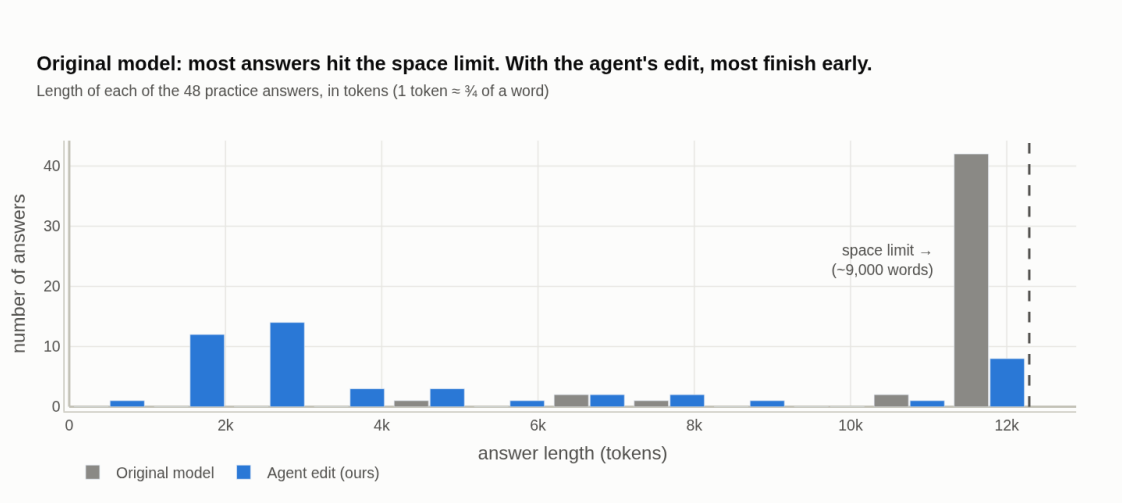


Figure 1: Length of each of 48 practice answers. Most of the original model's answers hit the space limit; with the agent's edit, most finish early.

Prompting does not help. A system prompt telling the model not to repeat itself does no better than a meaningless placebo prompt. Both actually *raise* the loop rate slightly (31% and 30%, against 23% without a prompt). The loop lives in the model's habits, not in its instructions.

The rules of the game

Setup

- **Model:** Qwen3.5-0.8B (Qwen Team, 2026), the same revision as the original study, run with llama.cpp (Gerganov et al., 2023).
- **Task:** grade-school math (GSM8K; Cobbe et al., 2021) with thinking on and a 12,288-token budget.
- **Edits allowed:** at most 4. Each edit picks a group of "neurons" (columns of an output matrix in one layer), a direction in the model's internal space, and a strength. It then changes only how strongly those neurons write along that direction:

$$w_j \leftarrow w_j - \alpha\,(u^\top w_j)\,u, \qquad |\alpha| \le 10.$$

In words: $\alpha = 1$ deletes that part of what the neurons write, $\alpha > 1$ flips it, and $\alpha < 0$ makes it stronger. Each edit is a rank-1 change.

- **Agents get:** the weights, a written brief that describes the defect (the loops, with an example), and a scorer with a fixed budget. The scorer offers 40 "quick scores", which measure how much the model *wants* to repeat itself without letting it write. It also offers 6 "test runs", in which the edited model really solves 16 practice problems.
- **Not allowed:** training, running the model outside the scorer, editing the vocabulary layers, or reading the original study's edit, our notes, other agents' work or the scorer's code. These rules were not enforced in software: we audited every action in every agent's log afterwards and found no violations.
- **Agents:** Claude Opus 5.5, one per run, about 2 hours each. Some runs also got an **atlas**: a detailed scan of the model's internal activity while looping.
- **How we judge:**
  - **Practice set:** we confirm each agent's pick on 48 other problems (dev48).
  - **Second practice set:** 32 test problems. We had already used them in exploratory runs, so they are not a sealed test.
  - **Locked test:** 200 test problems, opened once, after we wrote down our predictions. Three arms were added later and are marked as such.
  - **Harder math:** we check transfer to 148 competition problems (MATH-500; Hendrycks et al., 2021; Lightman et al., 2024) that no search ever touched.
- **Fair comparisons:** every method answers the same problems with the same random seeds, so we can count, problem by problem, who solved what.

Can an agent fix the loops by editing weights?

EXPERIMENT 1

Can we reproduce the published edit?

The published edit changes only two matrices, in layers 15 and 19. We recovered it exactly. Each change is a single rank-1 edit with strength 6: it takes the part of 64 neurons' output that lies along one "repetition direction" and flips it to $-5\times$ its original size. Both layers use almost the same direction, written by different neurons.

On our setup, the published edit lifts the practice set from 15% to 46%, and harder math from 8% to 21%. Our absolute MATH numbers are lower than the paper's because our token budget is 12k rather than 32k. All methods here share our budget. This matters for the published edit more than for the others: it is still improving when it hits 12k (see the locked test below).

EXPERIMENT 2

Can a Claude agent find its own fix?

Our first agent (a0) only had quick scores. It found a different edit, one that discourages the model from copying "Yes", "?" and "Wait" in two attention layers, and it matched the published edit on the second practice set (44% vs 41%).

Then we added the 6 test runs. **Two agents, working independently, converged on the same neurons and the same two-part recipe, and it is much stronger: 67% and 65% on the practice set.**

agent	what the edit does ($\leq$ 4 rank-1 edits)	practice set
a1	weakens the urge to say "Wait" in 3 neurons (L17n1915, L18n881, L22n573) and strengthens the `</think>` signal in the last layer	**67%** (26 rescued, 1 lost)
a2	flips what 4 neurons write along "Wait" (L16n998, L16n3422, and a1's L17n1915 and L22n573) and strengthens `</think>` in layer 22	**65%** (24 rescued, 0 lost)

Which a1 edit we report. The a1 agent ranked two variants of the same edit: the same neurons, with strengths -9 and -1.1 (its first pick) or -6 and -0.8 (its second). We carried the milder, second-ranked variant through every comparison because it scored higher on the practice set (67% vs 65%). That was our choice, not the agent's. On the locked test the two are indistinguishable (61.5% vs 62.0%), and the agent's first pick writes less than half as many tokens. "a1" below means the milder variant unless we say otherwise.

How does it do it: both agents read the weights "through the vocabulary". For each neuron, they asked which word it pushes the model to say. That is how they found neurons that push "Wait". Their notes show they learned from their failures. Negating those neurons outright made the model say "wait wait wait…" and solve nothing, which told them the neurons work in the opposite way from what they had first assumed.

The two parts only work together. Our ablations, each a test run on 16 problems:

- the `</think>` boost alone solves 19%;
- the "Wait" suppression alone solves 44–50%;
- both together solve 69%, and almost nothing runs out of space.

A loop needs a way back in ("Wait, let me check") and no way out. The edit closes the first and opens the second.

The same neurons, opposite signs. a1 and a2 share two neurons and the same "Wait" direction, but a1 makes their writing 7× stronger and a2 flips it to −9×. We checked what the neurons do on 12 looping transcripts of the original model:

- **a1 strengthens a brake.** Neuron L22n573 has a negative activation at every position just before a "Wait" (52 of 52), and its output weights point towards "Wait", so it pushes *against* the word. a1 amplifies that push. With the text held fixed, a1's three neurons lower the log-probability of "Wait" at those positions by 16.6 nats, and its last-layer edit alone raises `</think>` by 1.5 nats. The two parts are separable. L18n881 contributes almost nothing.
- **a2 reaches the same effect another way.** It also lowers "Wait" there (by 26 nats), but its flip of L22n573 alone would *raise* it. Its main lever is L16n998, which in the original model promotes "Wait" and after the edit suppresses it. The direct effect of the four neurons does not add up to 26 nats, so most of it must pass through later layers. We have not mapped that path.

So the agents agree on where to intervene, not on how. Figure 2 describes a1.

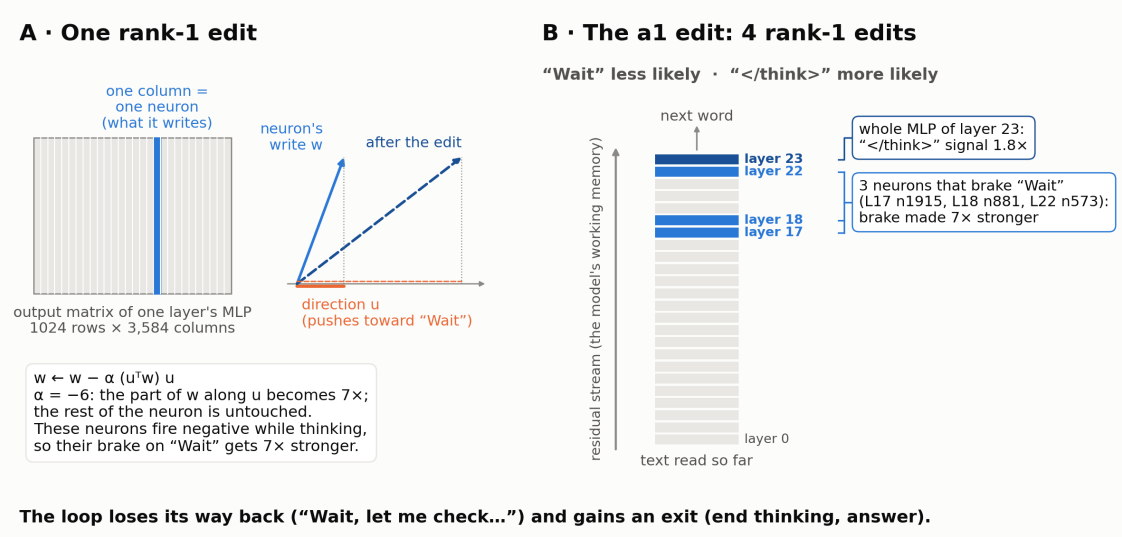


Figure 2: Anatomy of the a1 edit. (A) A rank-1 edit changes only the part of what a neuron writes along one chosen direction. (B) a1 strengthens three neurons' brake on "Wait" and the `</think>` signal in the last layer.

EXPERIMENT 3

Does it hold on problems it never saw?

On harder competition math, which was never used in any search, the edit solves 47%, against 21% for the published edit at the same 12k budget. Compared problem by problem with the published edit, it rescued 44 problems and lost 6. On the second practice set it solves 56% (the original: 13%; the published edit: 41%).

Two caveats on the MATH numbers. First, the published edit still runs out of space on 78% of these problems, so its 21% is limited by our budget (the original study reports 48% at 32k). Second, our answer checker is strict and misses some equivalent forms. Correcting the false negatives we found raises every method a little and keeps the order (a1 49%, a1 + time limit 55%, time limit 38%, published edit 22%). The tables use the uncorrected numbers.

The locked test. We opened 200 locked problems once, after writing down our predictions. At the full 12,288-token budget, a1 solves 61.5% (original 15.5%, published edit 53.5%). Problem by problem, it beats the published edit by **8.0 ± 3.4 points** (32 rescues, 16 losses; $p = 0.03$, uncorrected).

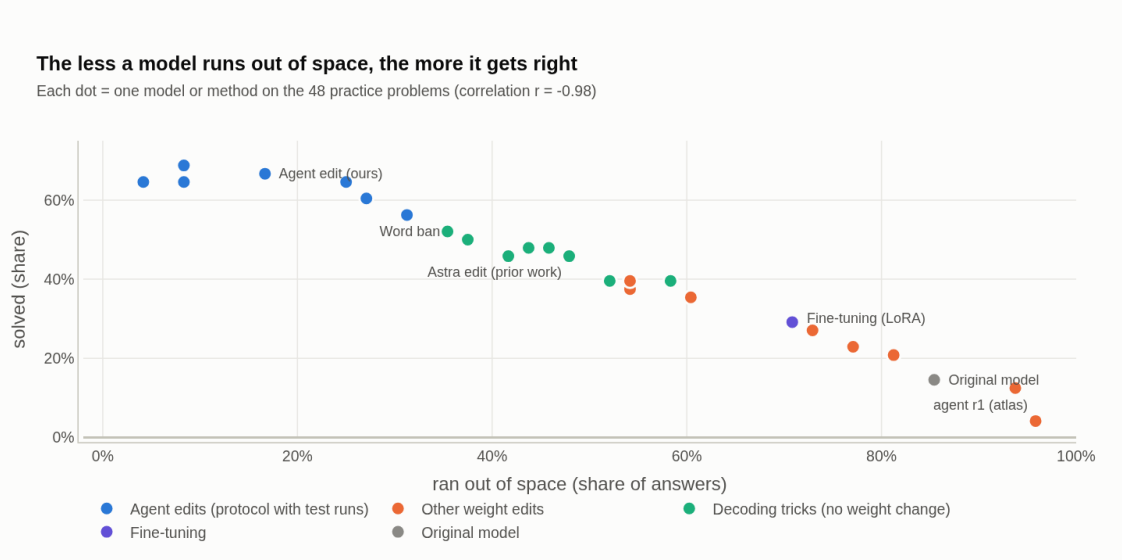


Figure 3: Across all 26 models and methods we ran on the practice set, the fewer answers that run out of space, the more are correct ($r = -0.98$).

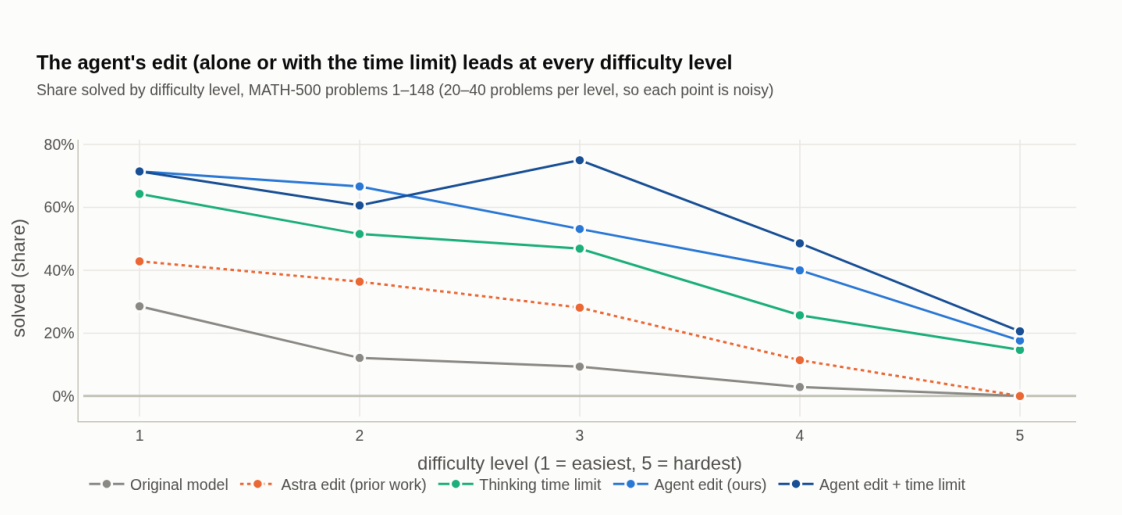


Figure 4: The agent's edit (alone or with a thinking time limit) stays ahead at every difficulty level.

But the budget decides that comparison. The table shows the share solved if we stop every answer at a smaller budget. Arms marked † were run after the test was opened.

method	2,048	4,096	6,144	8,192	12,288	cut off at 12k	mean tokens
original model	3.5%	6.5%	10.5%	12.0%	15.5%	84.5%	11,142
published edit (Astra)	10.0%	19.0%	24.5%	39.0%	53.5%	42.5%	8,877
fine-tuning (LoRA) †	12.5%	20.5%	26.0%	30.0%	33.0%	65.5%	9,354
ban the word “Wait” †	10.0%	26.5%	41.5%	50.5%	60.0%	35.5%	7,758
thinking time limit, 4,096	4.0%	6.5%	60.5%	60.5%	61.0%	34.0%	7,012
time limit, answer capped at 1,024 †	2.5%	5.0%	60.0%	60.0%	60.0%	35.0% ‡	4,596
agent’s edit (a1)	29.0%	45.0%	55.5%	60.5%	61.5%	17.0%	5,049
a1 agent’s first pick †	46.5%	60.0%	61.5%	62.0%	62.0%	5.5%	2,269
second agent’s edit (a2) †	37.5%	53.0%	63.5%	66.5%	67.5%	15.5%	4,387
edit from a Fable run (f1) †	12.0%	40.5%	52.5%	61.5%	66.0%	29.0%	6,534
a1 + time limit	25.0%	49.0%	67.5%	69.0%	69.5%	9.5%	4,081
a1 + time limit, answer capped †	25.5%	54.5%	69.5%	69.5%	69.5%	15.0% ‡	3,172
original model, thinking off †	—	—	—	—	58.5%	5.0%	1,263

‡ With a capped answer, this column counts answers stopped by the answer cap.

- **The published edit has not levelled off.** It gains 14.5 points between 8k and 12k, and 42.5% of its answers are still running at the cap. With a longer budget it would probably close some or all of the 8-point gap. We have not run that.
- **The agents’ edits level off early.** The agent’s first pick is at 60% by 4,096 tokens, where the published edit is at 19%. The supported claim is about efficiency: about the same accuracy with a third to a quarter of the tokens.
- **a2 is the best single edit (67.5%).** It beats the published edit by 14.0 ± 3.6 points (42 rescues, 14 losses). Its lead over a1 is 6 points (29 rescues, 17 losses; $p = 0.10$), and 15.5% of its answers are still cut off.
- **Switching thinking off is the cheapest fix on this benchmark.** The unedited model then solves 58.5% with about 1,300 tokens per answer. a1 with thinking on is 3.0 ± 3.5 points above it ($p = 0.48$): a tie. a2 is 9.0 ± 3.6 above ($p = 0.02$) and a1 with a time limit is 11.0

± 3.5 above ($p = 0.003$). With thinking off, the edits neither help nor hurt (a1 56.0%, published edit 57.5%).

All p-values are exact sign tests on discordant pairs, without correction for multiple comparisons. With a Holm correction over the comparisons in this section, a1 against the published edit is no longer significant; a2 against the published edit and a1 + time limit against thinking off still are.

Does another editor model find the same fix?

Run after the locked test was opened, under different conditions. Not comparable with a1 and a2.

We repeated the winning protocol (no atlas, same brief, same scorer, same budget) with two agents running Claude Fable 5.2 instead of Opus 5.5: f1 and f2. We report them as a replication of the idea, not as a comparison between editor models, because the conditions differed:

- both runs were interrupted twice by a subscription usage limit and resumed;
- they shared a slow scorer queue and first stopped on wall-clock time, after 4 of their 6 test runs;
- their prompt had extra rules (no web access, no reading of this paper's repository), because the answer was public by then;
- we extended both runs to the full 6 test runs after we had seen their first results.

Both runs fixed the loops with a different kind of edit. Both found the "Wait" neurons that a1 and a2 edited, and neither used them in its final answer. f1 flipped them, got "WaitWaitWait…" (the same first failure as a1 and a2) and moved on. Both ended with edits on every column of the last layers, along the direction of the token "Wait":

- **f1:** the MLP and attention outputs of layer 23 (2 edits);
- **f2:** the attention outputs of layers 20–23, with a small `</think>` component (4 edits).

In form these are closer to the published edit than to a1.

edit	practice set (48)	cut off	mean tokens	locked test (200)
f1	65% (25 rescued, 1 lost)	25%	6,711	66.0% †
f2	60% (22 rescued, 0 lost)	27%	6,618	—
a1, for reference	67%	17%	4,729	61.5%

- **Practice set.** Both tie a1 (f1: 6 rescues against 7; f2: 4 against 7). f1 beats the published edit (9 against 0).
- **Locked test.** f1 solves 66.0%, with 101 rescues and no losses against the original. It beats the published edit (40 rescues, 15 losses; $p = 0.001$) and ties a1 (28 against 19) and a2 (21 against 24).
- **f1 is more conservative.** 29% of its answers are still cut off and it writes more tokens, but 93% of its finished answers are right (a1: 74%; a2: 80%).
- **The extension did not help.** With two more test runs, f1 changed its pick to a variant that also suppresses "Yes" (11 of 16 on its own test run, against 9 of 16 before). On the practice set that variant solved 56%, below the earlier pick (2 rescues against 6). f2 kept its pick. A 16-problem test run cannot rank candidates this close, and the same caution applies to the choices inside a1 and a2. The numbers above are for the picks made before the extension.

This supports one thing: another editor model, given the same brief, also repaired the defect, by a third route. It does not support a ranking of editor models, or a claim that editors do not converge.

Is it just a trick?

A skeptic would ask whether the edit is a fancy way of doing something simpler. We tried the simpler things, tuned on 48 problems (three times what the agents saw), which favors them.

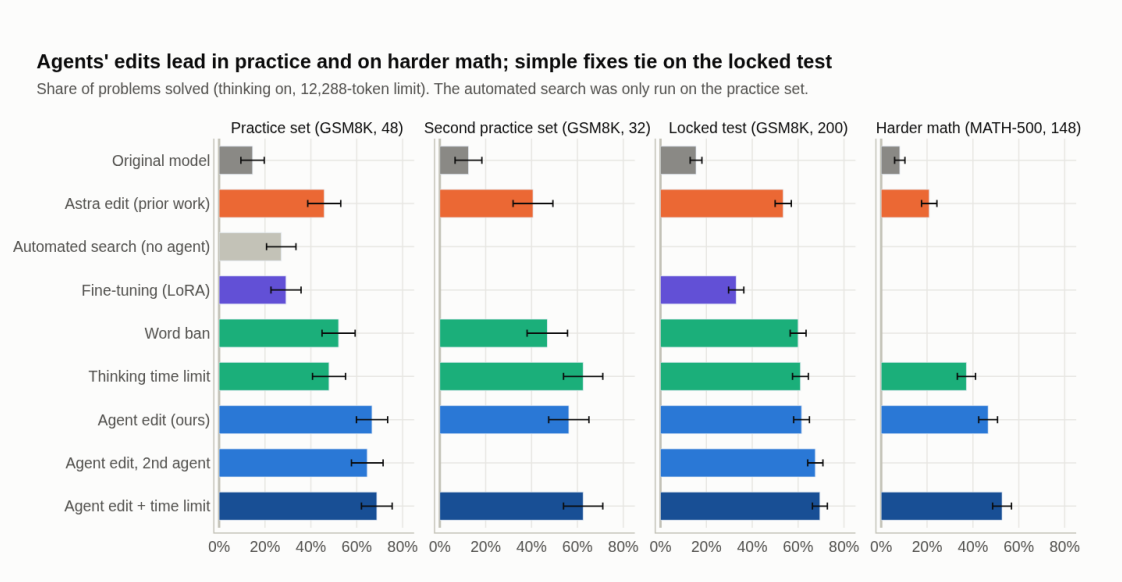


Figure 5: Share of problems solved by each method on the four problem sets (error bars: ±1 standard error).

method	practice set (48)	second practice set (32)	locked test (200)	harder math (148)
original model	15%	13%	15.5%	8%
original model, thinking off	—	—	58.5% †	—
published edit (Astra)	46%	41%	53.5%	21%
ban the word “Wait” (best of 3 settings)	52%	47%	60.0% †	27% (first 48)
thinking time limit, 4,096 tokens (best of 2)	48%	**63%**	61.0%	37%
time limit, answer capped at 1,024 (best of 3)	50%	—	60.0% †	—
mean-difference edit, no agent (best of 6)	—	31%	—	—
scripted search, no agent	27%	—	—	—
fine-tuning (LoRA)	29%	—	33.0% †	—
agent’s edit (a1)	**67%**	56%	61.5%	**47%**
a1 agent’s first pick	65%	—	62.0% †	—
second agent’s edit (a2)	65%	—	67.5% †	—
edit from a Fable run (f1), other conditions	65%	—	66.0% †	—
agent’s edit + time limit	**69%**	**63%**	**69.5%**	**53%**

† Run after the locked test was opened; no choice depended on these arms.

EXPERIMENT 4

Could a decoding setting do the same?

Setup

- **Word ban:** while the model writes, make "Wait" (and variants) nearly impossible and nudge `</think>`. No weight change.
- **Time limit ("budget forcing", Muennighoff et al., 2025):** after N tokens of thinking, cut it off and force an answer. No weight change.

Banning the word is worse on the practice sets and on harder math, and ties on the locked test. There it solves 60.0%, against 61.5% for a1 (25 rescues against 28), with 7,758 tokens per answer and 35.5% of answers cut off. We had predicted 40–52%. The edit is not a global ban. It works through neurons that are active when the model is about to restart, so it leaves "Wait" alone elsewhere.

The time limit ties the edit on the locked test (61.0% vs 61.5%; +0.5 ± 3.4). Our implementation of the limit looks more expensive than it is. After the forced `</think>`, some answers keep looping until the 12k cap, which is why 34% of them are cut off and the mean is 7,012 tokens. If we stop every answer at 6,144 tokens instead, the time limit keeps 60.5% at a mean of 4,910 tokens, about the same as a1 (5,049). So we do not claim that the edit is cheaper than a well-implemented time limit. We later ran that version, with the answer capped at 1,024 tokens after the forced `</think>` (thinking limit chosen among three on the practice set). It solves 60.0% at a mean of 4,596 tokens, the same accuracy as the uncapped limit (14 rescues against 16).

On harder math the edit is ahead of the time limit by 9.5 points, rescuing 23 problems and losing 9. Forcing an answer early works when the answer was already computed before the loop began, which is common in grade-school problems and rarer in longer ones.

Together they do best of all (53% on harder math and 69.5% on the locked test, 8 points above either alone, with few cut-off answers). The edit removes most loops, and the time limit catches what is left. An independent replication reports the same additivity on MATH subsets (tdotlab, 2026). With the capped answer, the combination keeps 69.5% on the locked test and drops to 3,172 tokens per answer. It is above a1 alone (26 rescues, 10 losses; $p = 0.01$), the capped limit alone (33 against 14; $p = 0.008$) and thinking off (37 against 15; $p = 0.003$).

Thinking off. We did not run thinking off on our MATH subset. Independent reruns report about 50% on the full MATH-500 with thinking off and a 32k budget (exp8-loop-controls, 2026), which is in the range of a1's 47–49% here. On this model, edits that repair thinking mostly recover what the model can already do without thinking; only the combination with a time limit, and a2, are clearly above that on the locked test.

EXPERIMENT 5

Could ordinary fine-tuning do the same?

Setup

- **Data:** the model's own reasoning on 600 *other* GSM8K problems. Only 57 attempts finished cleanly and correctly. For 279 that looped, we cut the reasoning where the loop began, closed the thinking and let the model answer: 118 answers were correct. That gives 175 training examples.
- **Training:** LoRA (Hu et al., 2022), a standard and cheap fine-tuning method (rank 16, all layers, 3 rounds). We picked the best round on the same 16 problems the agents used for test runs.

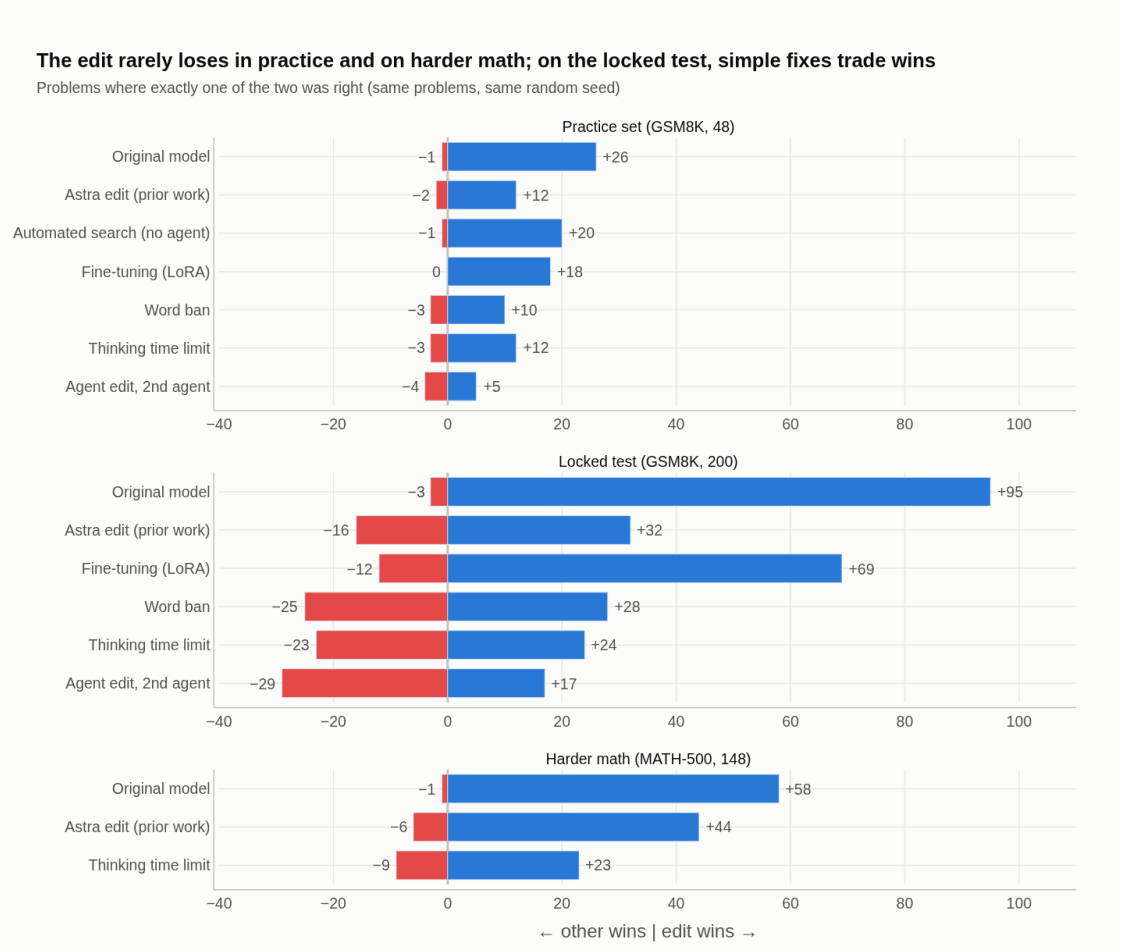


Figure 6: Problem by problem, same problems and seeds: problems only the agent's edit solved (right) against problems only the other method solved (left).

Fine-tuning reached 29% on the practice set. That is better than the original (15%), but problem by problem the agent's edit rescued 18 problems that fine-tuning missed and lost 0. The fine-tuned model still ran out of space on 71% of its answers (the edit: 17%). It learned what to write *after* a cut, not *when* to stop. This is one cheap recipe with 175 examples, and we did not tune it further. More data, a better data recipe or reinforcement learning might do better, at more GPU time (see below). The result is "this fine-tuning recipe lost", not "fine-tuning loses".

On the locked test the fine-tuned model solves 33.0% and still runs out of space on 65.5% of its answers. a1 rescues 69 problems it misses and loses 12.

EXPERIMENT 6

Is the agent needed, or just its tools?

We wrote a script that tries edits from the same toolbox the winning agents used: "word directions" for the 300 most common words, the neurons most aligned with them, random strengths. It had the same budget, 40 quick scores plus 6 test runs on the most promising candidates.

The script's best test run solved 3 of 16, and its pick reached 27% on the practice set, 40 points below the agents. With this toolbox and this budget, random choices were not enough.

This is a weak baseline, and stronger searches without an LLM exist:

- **Our own mean-difference edits.** We built six edits from the difference between the model's average internal state in looping and non-looping text, with no agent. The best one solved 31% of the second practice set, against 41% for the published edit and 13% for the original: about two thirds of the published edit's gain.
- **Concurrent work.** A mean-difference direction recovers 79% of the published edit's gain on MATH-500 (Zhang, 2026), and a linear program reaches about 93% (nathanrs, 2026). A blind repair with a weaker editor model and no description of the defect found nothing (WKJBryan, 2026).

So we do not claim that an agent is necessary. Our evidence is narrower: both agents beat every no-agent search *we* ran, and our brief told them what the defect was.

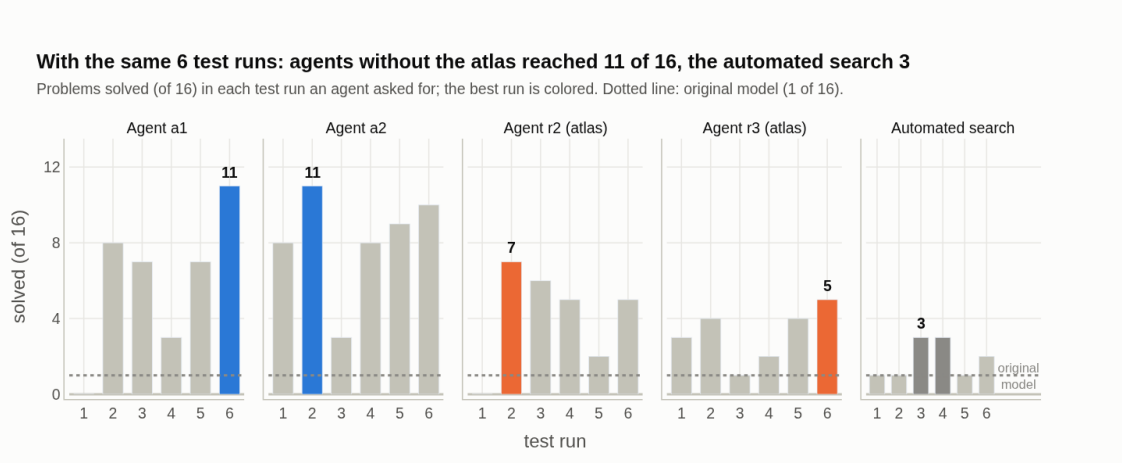


Figure 7: Each searcher's 6 test runs (16 problems each). The two agents without the atlas reached 11 of 16; the agents with the atlas 5–7; the script 3.

What we learned about the process

EXPERIMENT 7

Can a cheap score stand in for real tests?

Real test runs are slow (16 problems take about 10 minutes on our GPU), so the obvious move is a fast proxy. Every proxy we tried broke once a strong optimizer pushed on it:

- an edit that scored better than the published edit on the model's "surprise" at loop words gained 6 points in real use, against 28 for the published edit;
- an edit with no word-for-word loops at all still ran out of space 77% of the time;
- **agent r1 drove the quick score to 20× the published edit's value, and its edit solved 4% of problems, worse than doing nothing.** It blocked word-for-word copying, and the model simply looped in other ways (alternating values, growing lists).

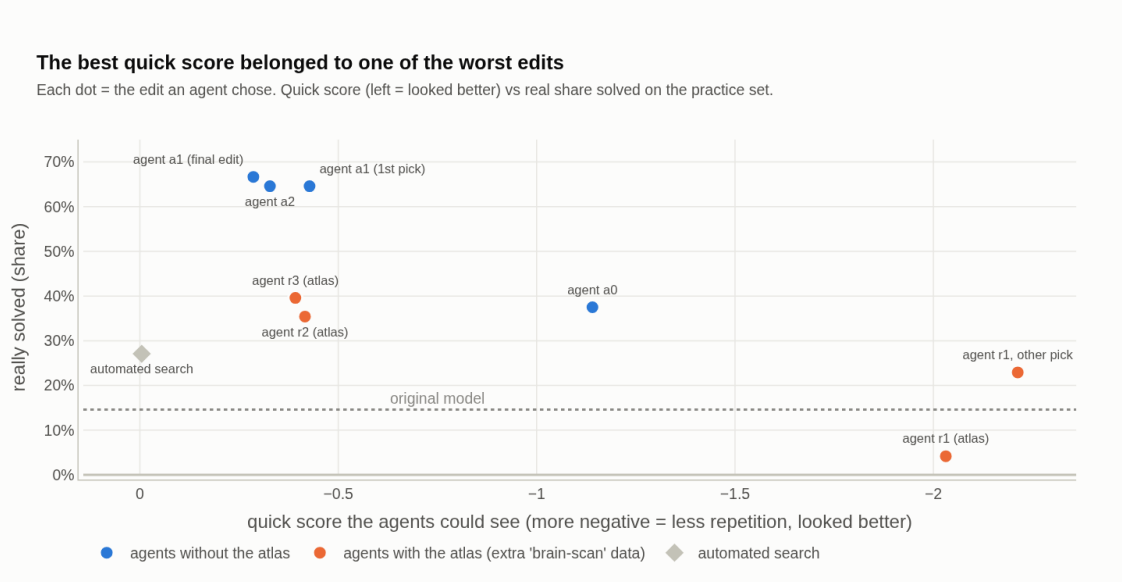


Figure 8: The quick score each agent's chosen edit achieved (left = looked better) against how often it actually solved problems.

Six real test runs fixed this. Under that protocol, both agents' best-scoring early ideas solved 0 of 16 in a test run, and the agents dropped them. **Our takeaway: an agent's objective must come from real behavior, even a small sample of it. Proxies are useful as guard rails, not as goals.**

EXPERIMENT 8

Does more interpretability data help?

We ran three pairs of agents with identical instructions; in each pair, one agent also had the atlas.

In all three pairs, the agent without the atlas found the better edit (38% vs 4%; 67% vs 35%; 65% vs 40%). With the atlas, agents chased intricate internal mechanisms, such as a "re-checking" state in the middle layers and "loop heads". These looked convincing in the data but did not stop the loops. Without it, agents went for the direct levers: the word "Wait" and the `</think>` signal. Three pairs make this suggestive, not proven. The first pair also ran under the older protocol without test runs, and we had predicted the opposite result for the first two pairs.

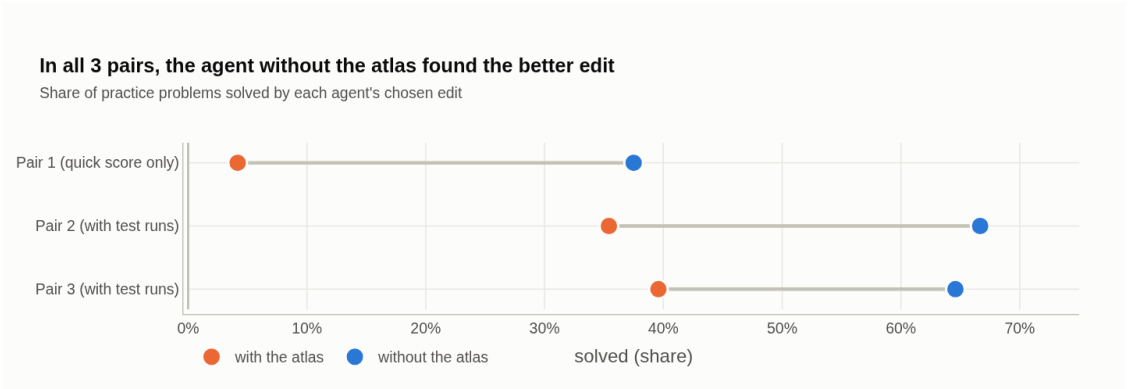


Figure 9: Each pair of agents had identical instructions; the agent without the atlas did better every time.

What did it cost?

	practice set	GPU-hours (1660 SUPER)	agent usage
agent's edit (one run)	**67%**	**1.3** (search only)	5.4 M tokens read, 48 k written (≈ $5–7 at API prices)
automated search	27%	1.6	—
tuning the decoding tricks	48–52%	4.4	—
fine-tuning (LoRA)	29%	17.1	—

- **One-time cost.** One agent run took about 2 hours, which is roughly the electricity of a microwave running for 15 minutes. Fine-tuning took 17 GPU-hours, 10 of them just to generate training data from a model that wastes most of its attempts.
- **Running cost.** Against the original model with thinking on, the edited model's answers are 60% shorter on the practice set, 55% shorter on the locked test and 29% shorter on harder math. The a1 agent's first pick is 80% shorter on the locked test, and a1 with a capped time limit is 72% shorter. This saving repeats on every question. It is a saving only against thinking on: with thinking off, the unedited model writes about 1,300 tokens per answer, less than any edit.
- **Compared with the original agent.** The published run used 86.8 M tokens read and 2.9 M written over 20 hours (from its public logs). Our agent used 16× and 60× less, in about 2 hours. The tools differ, so this is not a controlled comparison.
- **What the table leaves out.** The GPU-hours are for the search only. Confirming a pick on the practice set costs about 1 more GPU-hour for any method. The agent row is one successful run; we made six runs in total, and the three with the atlas did worse.
- **The fine print.** We ran on a subscription, so the dollar figures are list-price estimates. On a very cheap rented GPU, fine-tuning could cost a few dollars less than the agent's usage, for less than half the improvement.

Did the surgery break anything?

With thinking switched off, none of the edits changes math accuracy (200 problems) or tool-calling accuracy (300 requests) beyond about two standard errors. The edit acts where the loops happen, during thinking, and leaves other skills alone. One side effect is positive: with thinking on, the published edit gets better at declining irrelevant tool calls (16 of 16 vs 11 of 16), probably because shorter thinking leaves less room to hallucinate a call.

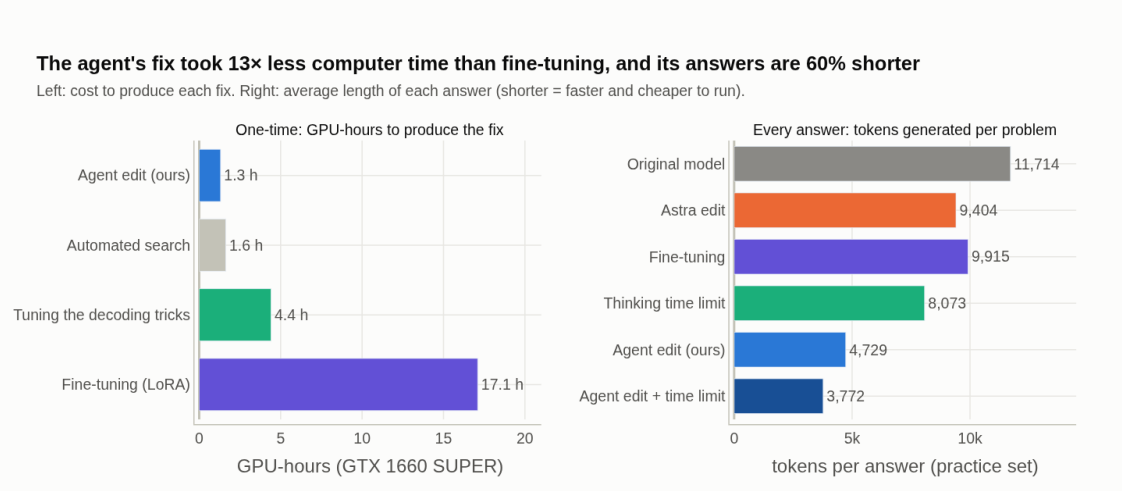


Figure 10: Left: GPU-hours to produce each fix. Right: average answer length, which sets the cost of every future answer.

One possible cost: finished answers are wrong more often. On the locked test, 74% of a1's finished answers are correct, against 91% for the published edit, 92% for the time limit and 97% for the original model. For the a1 agent's first pick, the strongest edit, it is 66%; for a2, 80%. Part of this is selection: the original model only finishes 31 of 200 problems, the easy ones, and the edits finish the hard ones too. But the order (stronger `</think>` push, lower accuracy when finished) suggests that the edits also make the model conclude before it is ready. We have not separated the two.

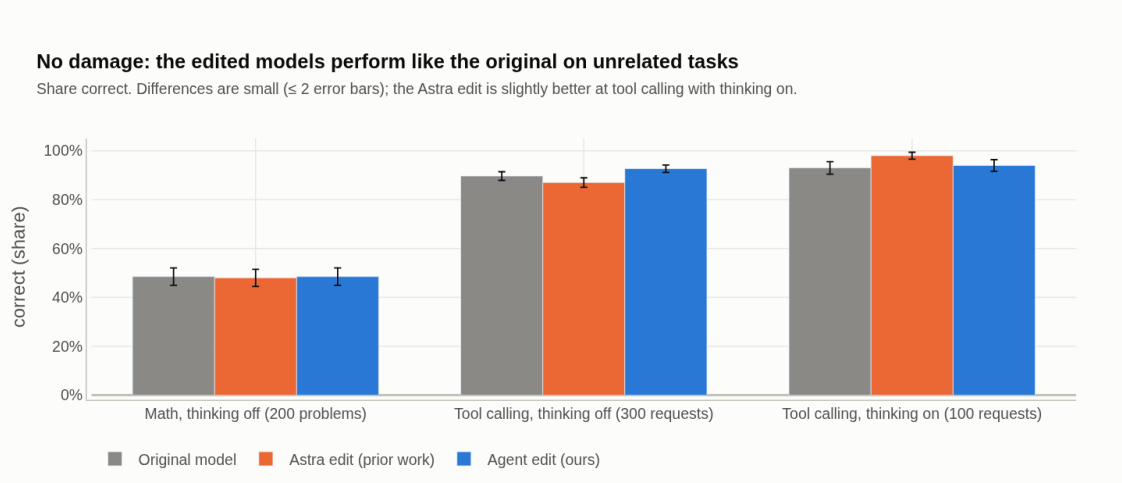


Figure 11: Accuracy on unrelated tasks for the original model, the published edit and the agent's edit.

Does it work on a second defect?

No, and the way it fails is informative. We needed a defect with no obvious keyword. Broken JSON and tool calls turned out not to be a problem for this model with thinking off: it was near ceiling. So we chose **instruction following** (IFEval, Zhou et al., 2023: "no commas", "all lowercase", "end with this exact phrase", "at least 3 highlighted sections"). With thinking off, the model follows every instruction in only about 55% of prompts. Its failures are spread over many instruction types.

Setup

- **Same protocol as for the loops:** an agent with 40 cheap probe scores and 6 real generation checks, at most 4 rank-1 edits.
- **Comparisons:** a system prompt asking for care, and LoRA fine-tuning on 414 of the model's own answers that passed the official checker (same recipe as before).
- **Confirmation:** on 152–200 IFEval prompts never used in any search.

arm	change vs original on fresh prompts
agent, 48-prompt checks (b1)	-3.3 ± 4.1
the loop edit (a1), not tuned for this	-3.9 ± 4.3
system prompt	-6.6 ± 4.2
fine-tuning (epoch chosen on 48 prompts)	-7.2 ± 4.3
agent, 128-prompt checks (b2)	-6.0 ± 3.7 (200 prompts)

Every method looked better in its practice check (+5 to +12 points) and lost on fresh prompts. The reason is sampling noise. Two runs of the *unchanged* model with different random seeds disagree on 23% of prompts. On 48 prompts, that gives a margin of error of 8–9 points, larger than any plausible gain. A larger, 128-prompt check did not rescue the agent either: the best edit's own opposite-sign control also looked slightly positive.

What this means. On the loops, the effect was +46 points and 16 problems were enough to see it. On instruction following, cheap changes of any kind (weights, prompt, gradient) seem to move the model away from a point that is already close to the best nearby, and no check of this size can find the rare changes that help. **The method's reach is set by the signal-to-noise ratio of the real test, not only by the agent.** One cheap fine-tuning recipe hits the same wall. No arm here differs from the original by more than two standard errors, so the result is "nothing helped", not "everything hurt". A second, sealed IFEval test is still unopened.

This does not contradict the IFEval gain reported for the published edit (21.0% to 62.3%; Takehi et al., 2026). That number is with thinking on, where three quarters of the unedited model's answers never close their thinking and score zero: it is the loop defect again. With thinking off, an independent rerun finds that the published edit does not help (-2.3 points; exp8-loop-controls, 2026), the same as what we see for a1 (-3.9).

The agent of the second run also found, from the weights alone, a direction in the residual stream that 99.7% of the decay-gate rows of all 288 DeltaNet heads (Gated DeltaNet, Yang et al., 2025) read with the same sign: a global "forgetting" control channel. Editing it changes the model, but not its instruction following. We leave it for future work.

Background and related work

Weight-space programming. Takehi et al. (2026) showed that frontier models can read and write neural network weights directly, including the loop-fixing edit we reproduce. We use their edit as our reference and their public logs for the cost comparison.

Model editing. Rank-one edits have been used to change facts stored in a model (ROME, Meng et al., 2022; MEMIT, Meng et al., 2023). Deleting a single direction can remove a behavior such as refusal (Arditi et al., 2024). Our edits belong to this family. The difference is that an agent chooses them by reasoning about behavior, instead of a fixed formula.

Steering. Adding a vector to the model's internal state at run time can steer its behavior (ActAdd, Turner et al., 2023; representation engineering, Zou et al., 2023). Our edits are permanent instead of applied at run time.

Overthinking. Reasoning models often re-check more than they need to (Chen et al., 2025). Known fixes include suppressing words like "Wait" (Wang et al., 2025) and capping thinking time (Muennighoff et al., 2025). We use both as baselines.

Agents and Goodhart. Agents already run interpretability experiments (MAIA, Rott Shaham et al., 2024). Strong optimizers are known to game proxy objectives (Gao et al., 2023; Skalse et al., 2022). We see the same with proxies derived from interpretability.

Concurrent work. *Sparks* drew fast follow-up, so far mostly preprints and public repositories without peer review. Hu et al. (2026) run 16 Astra editing trajectories on Qwen3.5-0.8B and 9B and report large held-out gains on MATH-500 and IFEval, all with thinking on and without decoding or fine-tuning baselines. Independent reruns of the Astra edit find that much of its gain is reachable without touching the weights: with thinking off and a long budget, the unedited model already scores about 50% on MATH-500 and 53–55% on IFEval (exp8-loop-controls, 2026; Zhang, 2026), and a difference-of-means direction found without an LLM recovers 79% of the MATH-500 gain (Zhang, 2026). A home replication with a Claude editor on a consumer GPU (tdotlab, 2026) independently located the same "Wait" brake neuron that our agents edited (layer 22, neuron 573), and found that its edit and a larger budget add roughly independently. Not every replication converges: MatinKhajavi (2026) reruns the experiments with Claude Opus 5.5 and gets edits of another kind (zeroing attention heads, plus the same neuron 573), and ajaygrover (2026) finds a Claude direction almost orthogonal to Astra's that stacks with it. Our two Opus agents agree on neurons but disagree on sign, and two later runs with another editor model left those neurons alone. This fits the picture: several different edits remove the loops. IWIBench (von Csefalvay et al., 2026) proposes a benchmark for editor models. Our contribution is the controlled comparison: agent edits against decoding, thinking-off and fine-tuning baselines on a locked test, with accuracy reported by token budget, compute accounting, evidence that agents game interpretability proxies, and a measured boundary where the method stops working.

What can't we claim yet, and what comes next?

- **Sample sizes:** the main comparisons now rest on a 200-problem locked test ($\pm$3–4 points), but dev and MATH numbers still rest on 48–148 problems.
- **One model, one defect that works:** "Wait" is an easy word to target, and the second defect (IFEval) gave a well-controlled negative result. **Next:** a concentrated defect without a keyword, and a second model. Any second defect needs a real test whose noise is well below the expected effect, measured before the search.
- **One token budget:** every comparison is at 12,288 tokens. The published edit is still improving there, so "a1 beats the published edit" is a statement about this budget. **Next:** both at a long budget on the locked test.

- **Thinking off is a strong baseline:** on the locked test it ties a1 and costs far fewer tokens. We have not run it on our MATH subset.
- **Late arms:** the a1 agent's first pick, a2, thinking off, the capped time limit, the word ban, fine-tuning and f1 were run on the locked test after it was opened. They are not pre-registered arms. The capped limit and f1 were each chosen on the practice set among two or three candidates.
- **Editor models:** the two Fable runs had different conditions from a1 and a2 and say nothing about which editor is better. **Next:** a ladder of editor models, including smaller ones, with several runs each under one protocol.
- **Noisy test runs:** the agents choose among candidates on 16 problems. The f1 extension shows that this cannot rank close candidates.
- **The a1 variant was our choice:** the agent ranked it second (see Experiment 2).
- **Mechanism:** we verified the brake story for a1 on 52 positions from 12 transcripts. a2's path is not mapped.
- **No-LLM searches:** our scripted search is a weak baseline, and concurrent work gets most of the published edit's gain without an LLM. **Next:** a targeted script (logit-lens directions, the same budget) against the agents.
- **Finished-but-wrong answers:** the edits lower accuracy among finished answers (see side effects).
- **Few replicates:** the winning protocol has 2 agents, and the atlas comparison has 3 pairs. **Next:** two more of each.
- **One fine-tuning recipe:** a stronger recipe could close part of the gap, at a higher cost.
- **MATH grading:** our checker has false negatives; the corrected numbers are in the text, not the tables.
- **Multiple comparisons:** p-values are uncorrected unless we say otherwise.
- **Honor system:** the agents' restrictions were audited, not enforced.
- **Different sampler from the original study:** absolute numbers shift, but every comparison here uses the same settings.

Predictions that failed. We wrote predictions in the notebook before each result. These were wrong:

1. The edit with the best quick score (r1) would beat the published edit. It solved 4%.
2. A weaker dose of r1's directions would recover the gain. It did not.
3. The atlas would help. It hurt in all three pairs.
4. The published edit would beat the thinking time limit on harder math. The time limit won (35% vs 21% on 48 problems).
5. The first IFEval agent would gain 5–9 points on fresh prompts. It lost 3.3 ± 4.1.
6. The second IFEval agent, with larger checks, would gain 5–15 points. It lost 6.0 ± 3.7.
7. a2 would score 52–64% on the locked test. It scored 67.5%.
8. At least one Fable run would edit the "Wait" neurons that a1 and a2 share. Neither did.
9. Banning the word would score 40–52% on the locked test. It scored 60.0%.

The claim, as it stands: for a concentrated behavioral defect with a large effect, an agent editing weights found a fix in about 2 hours on one consumer GPU. The fix is as accurate as the simple decoding alternatives, uses a third of the tokens of the published edit to get there, adds to a thinking time limit, and beat the one fine-tuning recipe we tried. For diffuse defects, such as instruction following in a 0.8B model, neither it nor cheap fine-tuning helped.

Data, code and logs

Everything is in the project repository:

- **Lab notebook:** dated, with predictions written before results (`docs/CADERNO.md`).
- **Records of every agent run:** briefs, notes, scorer logs, and every edit submitted (267).
- **Figures:** the notebook that draws every chart here (`notebooks/results.ipynb`).
- **Cost:** a detailed study (`docs/cost.md`).

The agents' full transcripts and the raw generations (~210 MB) will be released after review.

Use of AI tools

The editing agents studied here are Claude Opus 5.5, and Claude Fable 5.2 in two later runs. Claude (Opus 5.5 and Fable 5.2, through Claude Code) was also used as a research assistant throughout: it wrote most of the code, ran the experiments, kept the lab notebook, drafted this text and did an independent review of the claims against the raw data. The author designed the study, supervised and checked the work, and is responsible for the content.

Status of this draft and plan to submission

	status
Reproduction and dissection of the published edit	done
6 agent runs (2 protocols, 3 with/without-atlas pairs)	done
Decoding baselines, naive search, fine-tuning baseline	done
Transfer to harder math, side effects, cost	done
Locked test with $\geq$ 200 problems	done (5 pre-registered methods + 8 late arms)
More replicates (2 + 2 agents)	to do
Second defect without a keyword (IFEval)	done: negative, with fine-tuning and prompt baselines
Concentrated second defect; second model	to do (needed for a main-track venue)
Re-grade MATH with a symbolic checker	partly (false negatives corrected by hand; numbers in the text)
Edit-anatomy diagram	done (Figure 2)
Thinking-off baseline on the locked test (8k cap)	done (Nov 10): 58.5%, ties a1
Independent review of every claim against the data	done (Nov 10); this draft applies it
Mechanism check on activations	done for a1; a2 to do
Published edit vs a1 at a long budget	to do (5–8 GPU-h)
Time limit with a cap on the answer	done (Nov 11): 60.0% alone, 69.5% with a1
Two agent runs with another editor (Claude Fable 5.2)	done (Nov 11); different conditions, reported as a replication
Ladder of editor models under one protocol	to do (follow-up study)
Figures 5 and 6 with the new arms	done (Nov 11)
Verify citations	done (Nov 10; all 16 exist, 6 corrected, 5 added)

Venue: arXiv preprint first, then an interpretability or LLM-agents workshop with the current results, and a main-track conference if a concentrated second defect works.

Alternative titles: *Agents as Weight-Space Programmers: Fixing Reasoning Loops in Small Language Models*; *Generation in the Loop: What Makes Agentic Weight-Space Programming Work*.

Glossary

- **Language model, weights:** a model is a large collection of numbers (its *weights*) arranged in matrices. Qwen3.5-0.8B has about 800 million. Training changes them gradually, with gradients. Here, an agent changes a few of them directly and deliberately.
- **Token:** the piece of text the model reads and writes at a time, about ¾ of a word.
- **Thinking, loop:** before answering, the model writes a draft between `<think>` and `</think>`. It is *looping* when it repeats the same passage ("Wait, let me check…") until it runs out of space.
- **Rank-1 edit:** the smallest structured change to a matrix. It picks one direction and a group of *neurons* (matrix columns) and changes only how much those neurons write along that direction. Each fix used at most 4.
- **Fine-tuning, LoRA:** the usual route, which is training the model on examples. LoRA is the cheap version, which trains only small matrices added to the model.
- **Steering:** adding a vector to internal activations to push behavior. The Astra edit is of this kind, baked into the weights.
- **Decoding:** how text is sampled from the model's probabilities. *Word ban* (logit bias) and *thinking time limit* (budget forcing) change decoding, not weights.
- **Agent:** an AI model (here, Claude) working on its own with tools: it reads the weights, proposes edits, requests evaluations, and decides what to do next.
- **Quick score (probe) and test run:**
  - a quick score measures an internal probability in seconds (for example, how much the model "wants" to repeat);
  - a test run makes the edited model actually solve problems and checks the answers. Central lesson: quick scores can be gamed, and only test runs are reliable.
- **Practice set, locked test, pre-registration:** agents only saw 16 practice problems; we confirmed their picks on 48 others. The *locked test* stays sealed until every method has picked its final version. Predictions were written in the notebook *before* results were opened (pre-registration).
- **Standard error ($\pm$ SE), paired comparison:** "+8.0 $\pm$ 3.4" means a gain of 8 points with an uncertainty of about 3.4. A difference larger than ~2 SE is rarely luck. In a paired comparison, both methods solve the *same* problems: *rescues* are the ones only the first solves, and *losses* the ones only the second solves.
- **GSM8K, MATH-500, IFEval:** public benchmarks.
  - GSM8K: grade-school math.
  - MATH-500: competition math.
  - IFEval: following verifiable instructions ("no commas", "all lowercase"…).
- **Goodhart's law:** when a measure becomes a target, it stops being a good measure. This is what happened to the quick scores.
- **Atlas:** an extra map of the model's internal activity (which neurons fire during loops), given to half of the agents.
- **DeltaNet:** the layer type ("linear attention") that makes up 18 of Qwen3.5's 24 layers. It keeps a compressed memory of the text and has *gates* that decide how much to forget.

References

- ajaygrover (2026). weight-space-programming-stacking. GitHub repository, https://github.com/ajaygrover/weight-space-programming-stacking. Accessed Nov. 10, 2026.
- Arditi, A., Obeso, O., Syed, A., Paleka, D., Panickssery, N., Gurnee, W., Nanda, N. (2024). Refusal in Language Models Is Mediated by a Single Direction. NeurIPS 2024. arXiv:2406.11717.
- Chen, X., Xu, J., Liang, T., He, Z., et al. (2025). Do NOT Think That Much for 2+3=? On the Overthinking of Long Reasoning Models. ICML 2025, PMLR 267. arXiv:2412.21187.
- Cobbe, K., Kosaraju, V., Bavarian, M., Chen, M., et al. (2021). Training Verifiers to Solve Math Word Problems. arXiv:2110.14168.
- exp8-loop-controls (2026). Pre-registered controls for Experiment 8 of Sparks of Weight-Space Programming. GitHub repository, https://github.com/1099271/exp8-loop-controls. Accessed Nov. 10, 2026.
- Gao, L., Schulman, J., Hilton, J. (2023). Scaling Laws for Reward Model Overoptimization. ICML 2023. arXiv:2210.10760.
- Gerganov, G., and contributors (2023–). llama.cpp. Software. https://github.com/ggml-org/llama.cpp.
- Hendrycks, D., Burns, C., Kadavath, S., Arora, A., Basart, S., Tang, E., Song, D., Steinhardt, J. (2021). Measuring Mathematical Problem Solving With the MATH Dataset. NeurIPS 2021 Datasets and Benchmarks Track. arXiv:2103.03874.
- Hu, E. J., Shen, Y., Wallis, P., Allen-Zhu, Z., Li, Y., et al. (2022). LoRA: Low-Rank Adaptation of Large Language Models. ICLR 2022. arXiv:2106.09685.
- Hu, J., Tang, C., Li, W., Zhu, H., et al. (2026). Exploring Weight-Space Programming: Capabilities, Generalization, and Limitations. arXiv:2611.09382.
- Lightman, H., Kosaraju, V., Burda, Y., Edwards, H., et al. (2024). Let's Verify Step by Step. ICLR 2024. arXiv:2305.20050.
- MatinKhajavi (2026). Weight-Space-Programming: a replication with Claude Opus 5.5. GitHub repository, https://github.com/MatinKhajavi/Weight-Space-Programming. Accessed Nov. 10, 2026.
- Meng, K., Bau, D., Andonian, A., Belinkov, Y. (2022). Locating and Editing Factual Associations in GPT. NeurIPS 2022. arXiv:2202.05262.
- Meng, K., Sen Sharma, A., Andonian, A., Belinkov, Y., Bau, D. (2023). Mass-Editing Memory in a Transformer. ICLR

  2023. arXiv:2210.07229.
- Muennighoff, N., Yang, Z., Shi, W., Li, X. L., Fei-Fei, L., et al. (2025). s1: Simple test-time scaling. EMNLP 2025. arXiv:2501.19393.
- nathanrs (2026). weight-space-lp. GitHub repository, https://github.com/nathanrs/weight-space-lp. Accessed Nov. 10, 2026.
- Qwen Team (2026). Qwen3.5: Towards Native Multimodal Agents. Blog post, https://qwen.ai/blog?id=qwen3.5. Model: https://huggingface.co/Qwen/Qwen3.5-0.8B.
- Rott Shaham, T., Schwettmann, S., Wang, F., Rajaram, A., Hernandez, E., Andreas, J., Torralba, A. (2024). A Multimodal Automated Interpretability Agent. ICML 2024. arXiv:2404.14394.
- Skalse, J., Howe, N. H. R., Krasheninnikov, D., Krueger, D. (2022). Defining and Characterizing Reward Gaming. NeurIPS 2022. arXiv:2209.13085.
- Takehi, R., Teoh, J., Zhang, A., Khattab, O. (2026). Sparks of Weight-Space Programming: LLMs Can Natively Read & Write Neural Network Weights. Blog post, MIT CSAIL, Nov. 2, 2026. https://rikiyat.github.io/blog/weight-space-programming/. Logs: https://github.com/RikiyaT/lm_understand-logs.
- tdotlab (2026). weight-space-programming: a home replication with a Claude editor. GitHub repository, https://github.com/tdotlab/weight-space-programming. Accessed Nov. 10, 2026.
- Turner, A. M., Thiergart, L., Leech, G., Udell, D., Vazquez, J. J., Mini, U., MacDiarmid, M. (2023). Steering Language Models With Activation Engineering. arXiv:2308.10248.
- von Csefalvay, C., Doma, P., Unni Krishnan, A. (2026). IWIBench: Evaluating Gradient-Free Model Driven Intra-weight Injection. arXiv:2611.08647.
- Wang, C., Feng, Y., Chen, D., Chu, Z., Krishna, R., Zhou, T. (2025). Wait, We Don't Need to "Wait"! Removing Thinking Tokens Improves Reasoning Efficiency. Findings of EMNLP 2025. arXiv:2506.08343.

- WKJBryan (2026). wsp-lab. GitHub repository, https://github.com/WKJBryan/wsp-lab. Accessed Nov. 10, 2026.
- Yang, S., Kautz, J., Hatamizadeh, A. (2025). Gated Delta Networks: Improving Mamba2 with Delta Rule. ICLR 2025. arXiv:2412.06464.
- Zhang, Y. (2026). Rethinking Weight-Space Programming. Blog post and repository, https://github.com/ZhangYiqun018/rethinking-weight-space-programming. Accessed Nov. 10, 2026.
- Zhou, J., Lu, T., Mishra, S., Brahma, S., Basu, S., Luan, Y., Zhou, D., Hou, L. (2023). Instruction-Following Evaluation for Large Language Models. arXiv:2311.07911.
- Zou, A., Phan, L., Chen, S., Campbell, J., et al. (2023). Representation Engineering: A Top-Down Approach to AI Transparency. arXiv:2310.01405.